

# Железо для ИИ

От паспорта нагрузки к измеримому решению о машине.

## 1. Машину выбирают под работу

**Цель выпуска.** Рассчитать проверяемое требование к машине для обучения, инференса, локальных моделей и RAG. Числа в примерах учебные; совместимость и скорость подтверждаются опытом на точной модели.

Вопрос «какую модель запустит эта GPU?» неполон. Один и тот же checkpoint может загружаться в четырёхбитном формате для одиночного чата, не помещаться при длинном контексте и двадцати активных генерациях, а при полном обучении требовать совсем другой объём памяти. Модель также может *поместиться*, но работать медленнее допустимого из-за выгрузки слоёв в RAM.

Начните с паспорта: точная модель и ревизия, задача, формат весов, контекст, длина ответа, число активных последовательностей, целевые задержка и качество, размеры корпуса, операции записи и обновления, бюджет времени и денег. «20 пользователей» само по себе не равно двадцати одновременно удерживаемым KV-кэшам: это определяется поведением сервиса и планировщиком запросов.

**OCR и разбор PDF.** главный ресурс в типичном случае: CPU, RAM, SSD; иногда GPU для OCR; что измерять: Страниц/мин, ошибки извлечения, размер очереди.

**Эмбединги и reranking.** главный ресурс в типичном случае: GPU/CPU, RAM, пакетирование; что измерять: Документов/с, качество поиска, пик памяти.

**Интерактивная генерация.** главный ресурс в типичном случае: VRAM, bandwidth, KV-кэш, CPU для обвязки; что измерять: TTFT, inter-token latency, токенов/с.

**Пакетный инференс.** главный ресурс в типичном случае: GPU и планировщик, сеть/диск при подаче данных; что измерять: Суммарный throughput при заданном качестве.

**Полное обучение.** главный ресурс в типичном случае: GPU, HBM/VRAM, межузловая сеть, хранилище; что измерять: Токенов/с, шагов/с, восстановление после сбоя.

**LoRA/QLoRA.** главный ресурс в типичном случае: GPU-память под базу и активации, RAM/SSD; что измерять: Пик памяти, время шага, качество адаптации.

**Модель ограничений.** Для отдельного ядра Roofline связывает вычислительную мощность и пропускную способность памяти: при малой арифметической интенсивности ускорение вычислительных блоков почти не помогает. Для всей системы добавляются PCIe, сеть, декодирование корпуса и очереди. Это способ поставить гипотезу об узком месте, а не формула для точного предсказания токенов в секунду. FlashAttention показывает, что изменение перемещений между быстрой памятью чипа и HBM может улучшать attention без изменения его математической цели. См. [1–3].

## Выпуск 01

26 сентября 2026. Учебная глава к блоку VII, темам 7.1–7.12. Формулы дают оценки; конкретные checkpoint и runtime проверяются опытом.

## Что читатель сможет

Разложить нагрузку на этапы. Посчитать веса, KV-кэш и индекс.

Различить бюджеты обучения и инференса.

Составить паспорт и измерить результат.

## Вопросы к машине

Что запускаем? Сколько контекста и активных запросов? Какую задержку считаем допустимой?

# БИБЛИОГРАФИЯ

---

## ВВОДНЫЕ РАБОТЫ

1. NVIDIA (обновляемая документация), *CUDA Programming Guide*. Документация.  
Руководство по программированию CUDA  
Иерархия памяти, исполнение на GPU и связь аппаратных возможностей с программной моделью.  
*Что читать.* вводные разделы о programming model и memory; выписать, какие данные движутся между уровнями памяти.
2. Samuel Williams, Andrew Waterman, David Patterson (2009), *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*. Исследовательская статья.  
Roofline: наглядная модель пределов производительности  
Модель верхней границы производительности по compute и bandwidth.  
*Что читать.* определение operational intensity и границы применимости графика.

## СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Tri Dao et al. (NeurIPS 2022), *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. Исследовательская статья.  
FlashAttention: точное внимание с учётом обмена с памятью  
Пример проектирования алгоритма с учётом движения данных между SRAM и HBM.  
*Что читать.* постановку IO-сложности и схему tiling; не переносить показатели ускорения на любую GPU.

## 2. Память модели: веса, кэш и рабочие буферы

Нижние оценки весов, KV-кэша и рабочих буферов.

Первое приближение для плотной модели: сырые веса =  $P \times b / 8$  байт, где  $P$  — число параметров,  $b$  — бит на вес. Для 8 млрд параметров и 4 бит это 4 млрд байт, или примерно 3,73 GiB. Фактический файл и использование памяти отличаются: scales, zero-points, таблицы квантования, незаквантованные слои, runtime и временные буферы тоже занимают место. Количество *активных* параметров MoE за токен не равно общему объёму всех весов, которые нужно разместить или подгружать.

Для обычного полного KV-кэша decoder-only Transformer можно начать с

$$K = 2LH_{KV}DSBs,$$

где  $L$  — число слоёв,  $H_{KV}$  — число голов ключей/значений,  $D$  — размер головы,  $S$  — удерживаемые токены на последовательность,  $B$  — число одновременно удерживаемых последовательностей,  $s$  — байт на элемент. Множитель 2 означает key и value. При  $L=32$ ,  $H_{KV}=8$ ,  $D=128$ ,  $S=4096$ ,  $B=1$ ,  $s=2$  получаем 536 870 912 байт, то есть 0,5 GiB. Если таких последовательностей двадцать, сырая оценка — 10 GiB до накладных расходов менеджера кэша. GQA уже отражена в  $H_{KV}$ ; подстановка числа query heads зависит от оценки.

У sliding-window attention, MLA и иных архитектур другая схема хранения. Prefix sharing, квантизация и offload меняют одновременно объём и задержку. Кэш является функцией конкретной реализации, а не только названия модели; сверяйте код и документацию backend. Hugging Face документирует разные стратегии кэша, а PagedAttention и vLLM показывают, почему управление фрагментацией и совместное использование блоков важны для сервиса [4–5].

**Учебная таблица.** Числа ниже — только нижняя граница сырых весов в 4 битах. Это не минимально необходимая VRAM и не обещание запуска.

**1В.** сырые веса: 0,5 GB; что ещё меняет результат: Формат, поддержка архитектуры, буферы.

**3В.** сырые веса: 1,5 GB; что ещё меняет результат: Контекст и квантование.

**8В.** сырые веса: 4 GB; что ещё меняет результат: KV-кэш и длина ответа.

**14В.** сырые веса: 7 GB; что ещё меняет результат: Версия backend и буферы.

**32В.** сырые веса: 16 GB; что ещё меняет результат: Offload, несколько GPU, стоимость обмена.

**70В.** сырые веса: 35 GB; что ещё меняет результат: Размещение всех весов и обмен между устройствами.

**Практическая граница.** Установка `device_map=auto` или частичный GPU offload может помочь разместить модель, но перенос между RAM и VRAM меняет скорость. Не заключайте из успешной загрузки, что система удовлетворяет требованию к интерактивности [6].

### Перевод единиц

1 GB =  $10^9$  байт.

1 GiB =  $2^{30}$  байт.

### Три слоя оценки

Сырые веса; KV-кэш; буферы runtime и квантования.

### MoE

Различайте общие веса и активные параметры за токен.

# БИБЛИОГРАФИЯ

---

## ВВОДНЫЕ РАБОТЫ

4. Hugging Face (обновляемая документация), *Cache strategies*. Документация.  
Стратегии кэширования ключей и значений  
Dynamic, static, offloaded и quantized cache, а также ограничения отдельных реализаций.  
*Что читать*. сравнительную таблицу стратегий и примеры; записать версию библиотеки.
5. Woosuk Kwon et al. (SOSP 2023), *Efficient Memory Management for Large Language Model Serving with PagedAttention*. Исследовательская статья.  
Эффективное управление памятью при обслуживании LLM с PagedAttention  
Управление KV-кэшем блоками в обслуживании запросов.  
*Что читать*. источник фрагментации, отображение логических блоков и условия экспериментов.

## СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

6. Hugging Face (обновляемая документация), *Big Model Inference*. Документация.  
Инференс больших моделей  
Размещение частей модели между ускорителем, CPU и при необходимости диском.  
*Что читать*. device\_map, max\_memory и ограничения скорости offload.

### 3. Обучение требует другого бюджета

Почему полное обучение, LoRA и QLoRA требуют разных бюджетов.

Во время полного обучения к весам добавляются градиенты, состояние оптимизатора и активации для backward. Схема с FP32 master weights, BF16 параметрами, FP32 градиентами и Adam-состояниями даёт иной расход, чем чистое BF16, 8-bit optimizer или шардирование. Поэтому «18 байт на параметр» — пример конкретной схемы, а не физическая константа. На практике размер активаций зависит от microbatch, длины последовательности, архитектуры, checkpointing и параллелизма.

При DDP каждый участник обычно держит копию модели; FSDP и ZeRO разделяют параметры, градиенты и/или состояния оптимизатора между устройствами. Это экономит память ценой коммуникаций и сложности восстановления. Tensor parallelism делит операции внутри слоя; pipeline parallelism распределяет последовательность слоёв; context/expert parallelism решает иные задачи. Сравнивать конфигурации надо по общей пропускной способности при одинаковом качестве и допустимой стоимости, а не по числу карт [7–8].

LoRA обучает малое число адаптерных параметров. QLoRA хранит базовую модель в квантизованном виде и обучает адаптеры, но всё равно нуждается в памяти под активации, оптимизатор адаптера и временные тензоры. Существование эксперимента QLoRA на одной большой GPU не гарантирует, что любая модель и длина контекста поместятся на выбранной домашней карте [9–10].

**Восстановление — часть архитектуры.** Для длительного обучения важны скорость чтения данных, checkpoint, количество записей, достаточное место на SSD/object storage и тест восстановления после отключения. Максимальные FLOPS бессмысленны, если процесс регулярно простаивает на загрузке или теряет дни работы при сбое. В распределённой задаче паспорт должен содержать топологию сети и фактический объём обмена.

#### Память обучения

Веса, градиенты, оптимизатор, активации и временные тензоры.

#### Выбор метода

Полное обновление; адаптеры LoRA; QLoRA поверх квантованной базы.

#### Восстановление

Проверьте запуск после сбоя.

## БИБЛИОГРАФИЯ

---

### ВВОДНЫЕ РАБОТЫ

7. PyTorch (обновляемая документация), *Getting Started with Fully Sharded Data Parallel (FSDP2)*. Документация.  
Начало работы с полностью шардированным параллелизмом данных  
Разделение параметров, градиентов и состояния оптимизатора; применение к модели.  
*Что читать.* пример `fully_shard`, порядок `sharding` и сохранение состояния.

### СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

8. Samyam Rajbhandari et al. (SC 2020), *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. Исследовательская статья.  
ZeRO: оптимизация памяти для обучения крупных моделей  
Исходная постановка устранения избыточных копий состояния обучения.  
*Что читать.* уровни разделения и модель коммуникационных затрат.
9. Tim Dettmers et al. (NeurIPS 2023), *QLoRA: Efficient Finetuning of Quantized LLMs*. Исследовательская статья.  
QLoRA: эффективная адаптация квантованных LLM  
Адаптация поверх квантизированной базы, NF4 и другие механизмы снижения памяти.  
*Что читать.* схему памяти и ограничения экспериментов.
10. Hugging Face (обновляемая документация), *Quantization with PEFT*. Документация.  
Квантование и адаптеры PEFT  
Практическая реализация адаптеров поверх квантованной модели.  
*Что читать.* подготовку модели, целевые модули и ограничения совместимости.

## 4. Инференс: prefill, decode и очередь

Prefill, decode, очередь, пропускная способность и задержка.

Prefill обрабатывает входной контекст и создаёт начальные состояния; decode генерирует новые токены, многократно обращаясь к весам и кэшу. Для одного запроса эти фазы имеют разные профили работы. При пакетировании вес модели может обслуживать несколько последовательностей, но растут KV-кэш и ожидание в очереди. Управление блоками памяти, continuous batching, prefix caching и chunked prefill меняют компромисс между суммарной скоростью и задержкой отдельного пользователя [5, 11].

Измеряйте как минимум **TTFT** (до первого токена), **ITL/TPOT** (между токенами), end-to-end latency, суммарные output tokens/s, p95/p99 и долю ошибок/отказов. Для каждого опыта сохраните длину входа и ответа, частоту поступления запросов, политику batch, длину контекста, версию модели, драйвер и температуру устройства. Одинаковые средние tokens/s при разных хвостовых задержках могут означать различное качество сервиса.

**Учебный случай.** У нас 20 одновременно активных запросов с контекстом 4096 токенов. При параметрах раздела 2 сырой KV-кэш около 10 GiB. Если остальные 16 GiB заняты весами и буферами, карта с 24 GiB VRAM не обязана выдержать эту нагрузку. Возможны меньшая одновременность, иная архитектура, квантование кэша, больший ускоритель или очередь. Каждая мера меняет либо качество, либо задержку, либо цену; решение подтверждается тестом.

### Метрики сервиса

TTFT — до первого токена.

ITL/TPOT — пауза между токенами.

p95/p99 — хвостовые задержки.

### Равные условия

Модель, длины входа и ответа, частота запросов, batch, драйвер и backend.

## БИБЛИОГРАФИЯ

---

### ВВОДНЫЕ РАБОТЫ

11. vLLM (обновляемая документация), *Benchmark CLI*. Документация.  
Интерфейс измерения производительности vLLM  
Клиентское измерение TTFT, ITL и полной задержки.  
*Что читать.* смысл каждой метрики, генерацию нагрузки и экспорт результатов.

### СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

12. vLLM (обновляемая документация), *Documentation*. Документация.  
Документация vLLM  
Возможности планировщика, continuous batching, prefix caching и варианты параллелизма.  
*Что читать.* архитектурные функции и условия поддерживаемого backend; затем проверить применимость к своей модели.

## 5. Компоненты локальной машины и совместимость

Совместимость ускорителя, CPU, RAM, диска, платы и питания.

**GPU.** Смотрите сперва поддержку точной модели, формата и операций в нужном runtime; затем VRAM, пропускную способность памяти, численные форматы и устойчивую производительность. Версия CUDA и compute capability для NVIDIA, матрица ROCm с парой GPU/OC для AMD, Metal/MLX для Apple и XPU/SYCL либо иной backend для Intel определяют реальную применимость. Наличие общего GPU-бенчмарка не доказывает поддержку нужного kernel. Для AMD официальный список совместимости следует проверять по версии ROCm, а не по одному бренду карты [13–16].

**CPU и RAM.** CPU выполняет подготовку, OCR, токенизацию, индексирование, оркестрацию и часто часть offload. Важны потоки, пропускная способность RAM, PCIe lanes и стабильная скорость длительной работы. RAM должна вместить ОС, исходные данные, индексы, копии весов и буферы. Объём видеопамати и объём RAM нельзя бездумно складывать как одну быструю память.

**SSD, плата, питание.** Рассчитайте место на модели, датасеты, индексы, checkpoints, временные файлы и резервные копии. Проверьте сокет, BIOS, физические слоты и их линии, длину и толщину GPU, разъёмы БП и теплоотвод. Два слота на плате не обещают две карты на полной скорости. Для сервера важны сеть, управление доступом, резервное копирование и мониторинг, а не только пиковая генерация.

**Ноутбук/мини-ПК.** когда разумна: Один пользователь, ограничение места и питания; основная проверка: Доступная память, устойчивые частоты, поддержка backend.

**Настольная станция.** когда разумна: Локальные эксперименты, замена GPU/RAM, продолжительные задачи; основная проверка: Слоты, охлаждение, БП, шум, возможность расширения.

**Выделенный сервер.** когда разумна: Постоянная многопользовательская нагрузка; основная проверка: Надёжность, очереди, сеть, наблюдаемость, восстановление.

**Аренда ускорителя.** когда разумна: Редкое обучение или пиковый запуск; основная проверка: Полная стоимость, перенос данных, воспроизводимость окружения.

Этот список задаёт выбор архитектуры, но не продаёт конкретный SKU. Для закупки к нему добавляют таблицу точных компонентов, официальный лист совместимости и пробный запуск выбранного backend.

### Совместимость

Точный checkpoint

↓

Runtime и операции

↓

ОС и драйвер

↓

Конкретный GPU

### Кроме GPU

CPU, RAM, SSD, PCIe, плата, БП, охлаждение, сеть и резервные копии.

## БИБЛИОГРАФИЯ

---

### ВВОДНЫЕ РАБОТЫ

13. ggml-org (обновляемый код и документация), *llama.cpp*. Код и документация.  
llama.cpp: код и документация для локального инференса  
CPU, CUDA, HIP, Metal, Vulkan и другие backend, включая частичный offload.  
*Что читать.* supported backends, сборку и инструменты измерения; проверить конкретный commit.
14. AMD (обновляемая документация), *ROCm Compatibility Matrix*. Документация.  
Матрица совместимости ROCm  
Соответствие поддерживаемых ускорителей, ОС и версии стека.  
*Что читать.* выбранную версию матрицы перед покупкой AMD-карты.
15. NVIDIA (обновляемая документация), *Compute Capabilities*. Документация.  
Вычислительные возможности GPU NVIDIA  
Отображение моделей GPU на аппаратные возможности CUDA.  
*Что читать.* строку конкретного GPU и требования выбранного runtime.

### СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

16. Apple ML Research (обновляемый код), *MLX*. Код и документация.  
MLX: машинное обучение на Apple Silicon  
Вычисления на Apple Silicon с единой моделью памяти.  
*Что читать.* особенности unified memory и фактический расход задач; общий объём памяти не равен свободной памяти для модели.

## 6. RAG: корпус, индекс и база данных тоже требуют железа

Векторный индекс, корпус и генерация как отдельные бюджеты.

В RAG CPU и диск часто обрабатывают PDF, OCR и метаданные; GPU или CPU считают эмбединги; RAM и SSD держат поисковый индекс, текст и журнал обновлений; отдельный GPU может выполнять reranking и генерацию. Ускорение LLM само по себе не устраним медленную индексацию или плохую разметку документов.

Для  $N$  векторов размерности  $d$  из  $s$  байт на компонент сырые значения занимают  $N \times d \times s$ . Учебный пример: 150 000 фрагментов  $\times$  1024 измерения  $\times$  4 байта = 614 400 000 байт, или 0,572 GiB. В реальности добавятся структура HNSW/IVFFlat, первичный текст, метаданные, индексы фильтров, журнал, копии и временная память на построение. Из «3000 PDF» нельзя вывести  $N$ : сначала нужна выборка по страницам, OCR и правило разбиения.

pgvector предоставляет точный поиск по умолчанию и приближённые индексы HNSW/IVFFlat; выбор влияет на скорость, recall, время сборки и память [17]. Для академического или корпоративного корпуса проверьте не только среднюю задержку top-k, но и качество ссылок на точные страницы, фильтры доступа, обновление версии документа и восстановление после сбоя.

**Три независимых бюджета:** хранилище исходников и резервных копий; RAM/SSD индекса и метаданных; VRAM и вычисления моделей. Их можно размещать на одном компьютере или разделять по машинам. Решение зависит от числа одновременных запросов и от того, какие данные можно передавать наружу.

### Три бюджета

Исходники и копии.

Индекс, текст и метаданные.

Модели, KV-кэш и вычисления.

### Качество

Страница цитаты, права доступа, версия документа, recall и задержка.

## БИБЛИОГРАФИЯ

---

### ВВОДНЫЕ РАБОТЫ

17. pgvector (обновляемая документация), *README: Indexing and Exact/Approximate Search*. Код и документация.

pgvector: индексация, точный и приближённый поиск

Типы индексов, выборка и параметры памяти/поиска.

*Что читать.* exact search, HNSW, IVFFlat и примеры измерения размеров индекса.

## 7. Паспорт нагрузки и решение о покупке

От требований и измерений к обоснованной конфигурации.

Ниже шаблон, который надо заполнить **до** выбора конфигурации. Его машиночитаемая версия и учебный калькулятор приложены к выпуску. Калькулятор считает только нижние оценки трёх массивов и не выводит «совместимо».

**Задача.** пример записи: RAG по 3000+ PDF/Excel, один исследователь или 15 членов команды.

**Модели.** пример записи: Точные ID и ревизии генератора, embedding, OCR и reranker.

**Нагрузка.** пример записи: Распределение входных/выходных токенов, частота и активные генерации.

**Качество.** пример записи: Набор вопросов, точность ссылок, допустимая потеря после квантизации.

**Сервис.** пример записи: TTFT, ITL, p95, допустимая очередь и время восстановления.

**Данные.** пример записи: Страницы, фрагменты, размерность, история версий, резервные копии.

**Стек.** пример записи: ОС, драйвер, runtime, формат весов, настройки batch и KV.

**Цена.** пример записи: Покупка, электричество, хранение, поддержка, аренда на пике.

**Порядок выбора.** (1) Получить baseline на имеющемся оборудовании. (2) Оценить память с запасом и проверить поддержку backend. (3) Назвать 2–3 допустимых класса машины, включая аренду, если обучение редко. (4) Протестировать точную модель и корпус на целевых длинах и конкурентности. (5) Выбирать по качеству, p95 и полной цене; протокол сохранить рядом со спецификацией. Для ноутбука с 6 GiB VRAM разумный первый опыт — маленькая модель и короткий контекст, а не заранее заявленная поддержка 8B при любой квантизации.

**Контроль отказа.** Если запуск падает, различите отсутствие поддержки операции, OOM весов, рост KV, нехватку RAM, ограничения файловой системы и неправильную сборку. Если работает медленно, разделите prefill/decode, проверьте CPU offload, использование памяти и частоты под длительной нагрузкой. Изменяйте один фактор за опыт.

### Паспорт

Checkpoint и ревизия.

Активные запросы.

Контекст и ответ.

Корпус и индекс.

Задержка, качество, полная стоимость.

### Решение

Измерить baseline, проверить совместимость, оценить память, испытать нагрузку, сохранить спецификацию.

## 8. Практикум и исследовательский фронтир

Опыты на реальной нагрузке и направления дальнейшего исследования.

**Опыт А — память.** Возьмите 1–3B и 7–8B модели с опубликованными параметрами архитектуры. Предскажите размеры весов и KV для 1, 4 и 20 последовательностей; запишите остаток памяти после загрузки и пик при генерации. Объясните расхождение с формулой.

**Опыт В — скорость.** На одном checkpoint сравните CPU, GPU и частичный offload при фиксированных входах 512/4096 токенов и выходах 128/512. Сохраните TTFT, ITL, p95, энергопотребление и версии стека. Не сравнивайте throughput разных моделей как доказательство превосходства железа.

**Опыт С — RAG.** Измерьте реальные страницы и фрагменты выборки, посчитайте индекс, время OCR, embedding и поиска. Сравните точный поиск с HNSW при одинаковом наборе вопросов и целевом recall; проверьте страницы цитат.

**Опыт D — обучение.** На небольшой модели сравните полный шаг, LoRA и QLoRA при одинаковых данных и оценке качества. Зафиксируйте peak VRAM, шагов/с, размер checkpoints и возможность восстановления.

Фронтирные темы для продолжения: разделение prefill и decode между устройствами; disaggregated serving; точность и цена квантизации KV; динамическая маршрутизация MoE; совместное размещение поиска и генерации; отказоустойчивое многопользовательское обслуживание; алгоритмы с меньшим движением данных; согласованное измерение энергии на задачу. Каждое улучшение проверяется при одинаковом качестве и полной стоимости, а не только по одному бенчмарку.

Для телеметрии GPU можно использовать официальные счётчики DCGM, учитывая поддержку поля конкретным устройством [18]. Результат выпуска — паспорт нагрузки, файл расчёта, совместимая спецификация, журнал пробного запуска и обоснованное решение по классу машины. Следующий выпуск сможет сравнить конкретные компоненты и цены на дату закупки.

**Границы выпуска.** Проверены указанные официальные страницы, репозитории и страницы работ; глубина чтения отдельных академических текстов варьирует от аннотации до разделов, о чём сказано в журнале выпуска. Числа в примерах вычислены из формул и не являются замерами на реальной машине. Конкретная корзина комплектующих, совместимость каждого checkpoint и рыночные цены требуют отдельного протокола испытаний.

### Четыре опыта

- A. Память.
- B. Скорость CPU/GPU/offload.
- C. Индекс и поиск RAG.
- D. Обучение, LoRA, QLoRA.

### Сравнение

Одинаковые данные и качество; версии стека; p95 и цена задачи.

## БИБЛИОГРАФИЯ

---

### СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

18. NVIDIA (обновляемая документация), *DCGM Exporter Metrics*. Документация.  
Метрики экспортера DCGM  
Метрики использования памяти, температуры, мощности и ограничений GPU.  
*Что читать.* семантику полей и поддержку профилирования на выбранном устройстве.